CS54 DATA COMMUNICATION & NETWORKING · MSRIT · INTERNAL TEST 1

THE FINAL 10

Ten questions, full answers. Study these and walk in.

8 / 10	3 / 5 / 2	3	30
are literally the Apr 2025 CIE-1 paper, part for part	CO1 / CO2 / CO3 — matches how Q1 is built	numericals: Q5, Q9, Q10. Do these first in the hall	marks. Q1 compulsory, then Q2 *or* Q3

1 Recursive and iterative DNS resolution

CO1 · IN BOTH OF THE LAST TWO CIE-1S · 5 MARKS · DRAW IT

Setup: host `cse.nyu.edu` wants the IP of `gaia.cs.umass.edu`; its local server is `dns.nyu.edu`.

Iterative — the real case. 5 steps, 8 messages

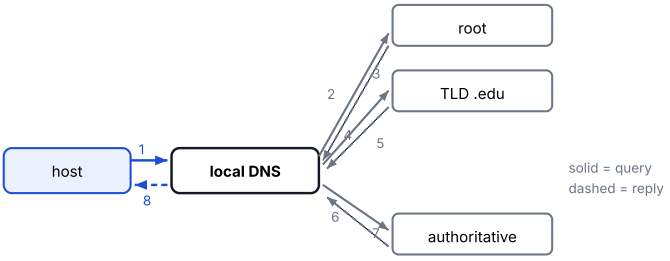
1. Host → local DNS server *(this first hop is always **recursive**)*
2. Local → **root**; root returns the IP of the **.edu TLD** server
3. Local → **TLD**; returns the IP of the **authoritative** server `dns.umass.edu`
4. Local → **authoritative**; returns the IP of the host
5. Local DNS server → host

Each server says *"I don't know, but ask this one"* — so the **local server does all the work. 8 messages = 4 queries + 4 replies.**

Recursive

host → local → root → TLD → authoritative, **each server querying the next on behalf of the previous**; the answer travels back down the same chain. Load shifts **upward** onto the higher-level servers.

ITERATIVE



Recursive	Iterative
"Find me the **complete answer**" — the receiving server is responsible for getting it	"If you don't know, **tell me whom to ask next**" — returns the best info it has
More load on **root / TLD** servers	Work done by the **local DNS server**

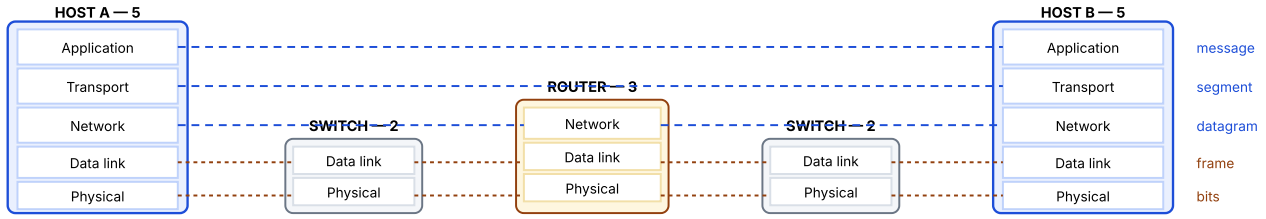
LAST MARK Add: *"DNS caching at the local server lets steps 2–4 be skipped next time, until the TTL expires."* In the real Internet it is **hybrid**: host→local recursive, everything after that iterative.

2 TCP/IP protocol suite — the 5 layers and their logical connections

CO1 · 3 CIE-1S: 2025, 2023, 2021 · 5 MARKS · DRAW IT

Open with: built by **DARPA in the 1970s for ARPANET**. A **hierarchical** protocol suite — each upper-level protocol is supported by lower ones. Originally **4 software layers on hardware**; today a **5-layer** model.

Layer	Contribution to the communication	Object	Address
5 Application	Process-to-process request/response. The user's message is created here. HTTP, SMTP, DNS, FTP.	Message	hostname
4 Transport	**End-to-end** logical connection; wraps the message in a **segment (TCP)** or **user datagram (UDP)**; delivers to the right process; TCP adds reliability.	Segment	Port, 16 bit
3 Network	**Host-to-host** delivery of datagrams; adds source and destination IP; routers choose the best route. IP is connectionless, best-effort.	Datagram	IP, 32 bit
2 Data link	Moves frames **hop to hop** across each link; adds the MAC of the next device; detects errors on the link.	Frame	MAC, 48 bit
1 Physical	Carries the bits as electrical / optical / wireless signals over the medium.	Bits	—



Logical (dashed) connections between peer layers — draw this

Logical connection = an imaginary direct link between the **same (peer) layer** at two devices. **Only the physical layer really moves the bits.**
End-to-end: application, transport, network — domain = **the internet**.
Hop-to-hop: data link, physical — domain = **a single link**. A **hop** is a host or a router.

HOOK **Host 5, Router 3, Switch 2.** Objects top-down: **M-S-D-F-B** = "**M**y **S**ister **D**raws **F**unny **B**irds".

IF IT ADDS "NETWORK LAYER SERVICES" logical addressing, routing, packet forwarding, internetworking — and IPv4 is **connectionless, best-effort**.

3 Cookies — keeping user state over stateless HTTP

CO1 · CAME IN CIE-1 APR 2025 · 5 MARKS · DRAW IT

Why: HTTP is **stateless** — the server forgets the client the moment it replies. Cookies let a site identify a user **across requests and across visits**.

The 4 components — write all four

1. A `Set-cookie:` header line in the HTTP **response**
2. A `Cookie:` header line in subsequent **requests**
3. A **cookie file** on the user's machine, managed by the browser
4. A **back-end database** at the website

Amazon login — 6 steps

1. User logs in: browser sends `POST /ap/signin` with username and password.
2. Server creates a **unique ID** (`session-id = ABC123`), stores it in its **database** against this user, replies `Set-cookie: session-id=ABC123`.
3. Browser saves it in its **cookie file** (`amazon → ABC123`).
4. On every later request the browser **automatically** adds `Cookie: session-id=ABC123`.
5. Server looks the ID up, recognises the user, returns **personalised content** — no password again.
6. A week later the cookie is still sent (until it expires), so the user is still recognised.



The timeline the examiner wants

HOOK **Set it, Send it, Store it, Search it** = Set-cookie header · Cookie header · browser cookie file · back-end database.

SCORING KEY "HTTP is stateless" opening line (1) · the 4 components (1.5) · the timeline diagram (1.5) · the numbered example (1).

4 Go-Back-N with a lost frame (+ the Selective Repeat window)

CO2 | 3 CIE-1S: 2025, 2024, 2021 | 5 MARKS | DRAW IT

1. Idea

GBN is a reliable-data-transfer (**ARQ**) protocol. The sender may send up to **N** packets without waiting for ACKs. If a packet is lost, the sender **goes back** and retransmits that packet **and every packet after it**.

2. Sender

Window of size N with **base** (oldest unACKed) and **nextseqnum**. **ONE timer**, for the oldest unACKed packet. **Cumulative ACKs** — "ACK n" means all packets up to n arrived. On timeout it resends `base … nextseqnum−1`.

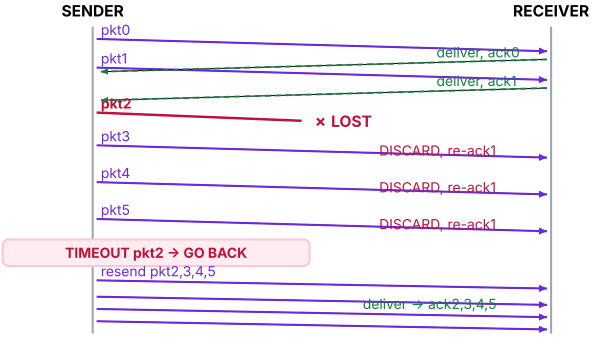
3. Receiver

Accepts **only the expected in-order packet** (delivers + ACKs it). Any out-of-order or corrupted packet is **DISCARDED** and the ACK of the **last in-order** packet is re-sent. **The receiver has no buffer.**

5. Drawbacks

- Correctly received packets **after** the loss are thrown away and resent → **wasted bandwidth**.
- Poor on **noisy networks** — one loss costs a whole window.

4. Sequence diagram, N = 4, packet 2 lost



IF IT ASKS SR WINDOW = 2^{M-1} The receiver **buffers out-of-order packets**, so sender and receiver windows must **never overlap**: $W_s + W_r \le 2^m$ → **$W \le 2^{m-1}$**. With m = 2: W = 3 fails (all ACKs lost, resent pkt 0 is accepted as *new* data — an error); W = 2 works (the resent 0 is outside the receiver's window 2,3, so it is recognised as a duplicate and re-ACKed).

HOOK GBN = **one timer, no buffer, cumulative ACK, resend everything.** SR = **many timers, buffer, individual ACK, resend one.**

5 RTT, DevRTT and the TCP timeout interval — numerical

CO2 | IN BOTH OF THE LAST TWO CIE-1S | 5–6 MARKS | FREE MARKS

Write all three formulas before substituting — that line alone is worth a mark.

$$\text{EstimatedRTT} = 0.875 \cdot \text{old} + 0.125 \cdot \text{Sample} \quad (\alpha = 0.125)$$

$$\text{DevRTT} = 0.75 \cdot \text{old} + 0.25 \cdot |\text{Sample} - \underline{\text{new}}\ \text{EstimatedRTT}| \quad (\beta = 0.25)$$

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

Apr 2025 — Sample 170 ms, old Est 150, old Dev 20

```
Est = 0.875 × 150 + 0.125 × 170
    = 131.25 + 21.25          = 152.5 ms

Dev = 0.75 × 20 + 0.25 × |170 − 152.5|
    = 15 + 4.375              = 19.375 ms

TO  = 152.5 + 4 × 19.375
    = 152.5 + 77.5            = 230 ms
```

May 2024 — two samples 106 then 120 (start 100 / 5)

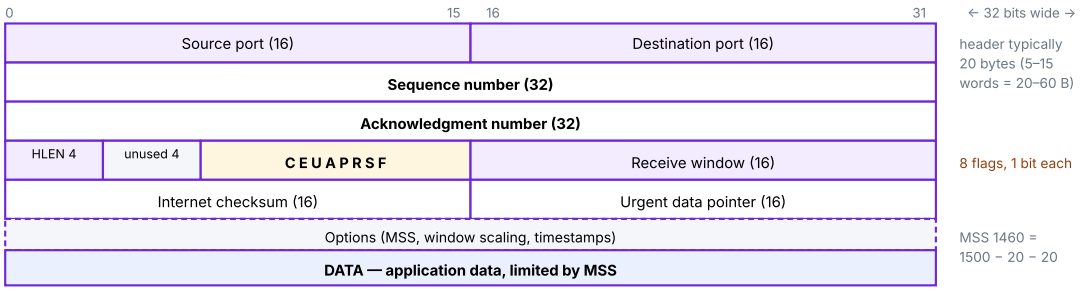
```
— Sample 1 = 106 —
Est = 0.875×100 + 0.125×106   = 100.75
Dev = 0.75×5 + 0.25×5.25      = 5.0625
TO  = 100.75 + 20.25           = 121 ms

— Sample 2 = 120 (uses S1's results) —
Est = 0.875×100.75 + 15       = 103.156
Dev = 3.797 + 4.211            = 8.008
TO  = 103.156 + 32.031         = 135.19 ms
```

3 TRAPS THAT COST MARKS EVERY YEAR (1) Compute **EstimatedRTT first** — DevRTT uses the **new** value, not the old one. (2) The second sample uses the **first sample's results** as its "old" values. (3) It is **4 × DevRTT**, never 2 or 3. Always show the substitution line and the units.

6 TCP segment structure and any four flags

CO2 | CIE-1 APR 2025 & JUN 2023 | 5 MARKS | DRAW IT



- **Source / destination port (16 each)** — identify the sending and receiving **processes**
- **Sequence number (32)** — byte-stream number of the **first data byte**. TCP numbers **bytes, not segments**
- **Acknowledgment number (32)** — **next byte expected**; valid when ACK = 1
- **Header length (4)** — header length in 32-bit words · **Unused (4)** — reserved, 0
- **Receive window (16)** — bytes the receiver will accept → **flow control**
- **Internet checksum (16)** — over **header and data** · **Urgent pointer (16)** — valid only if URG = 1
- **Options** (variable) · **Data** — the application data

Flag	Meaning
SYN	Establish a connection; synchronises the **initial sequence numbers** (step 1 of the handshake)
ACK	The ack-number field is **valid**; set in almost every segment after setup
FIN	Sender has finished and wants an **orderly close**
RST	Error condition / **abort** the connection immediately

SEQ/ACK EXAMPLE Seq = 100 carrying "HELLO" holds bytes **100–104**; the server replies **ACK = 105**. **ACK = last byte received + 1.**

HOOK C E U A P R S F = "**C**ute **E**lephants **U**sually **A**dmire **P**urple **R**oses, **S**unflowers, **F**igs".

7 Three-way handshake and connection termination — significance of each step

CO2 | CIE-1 APR 2025 & JUN 2023 | 5 MARKS | DRAW IT

A. Establishment — 3 segments

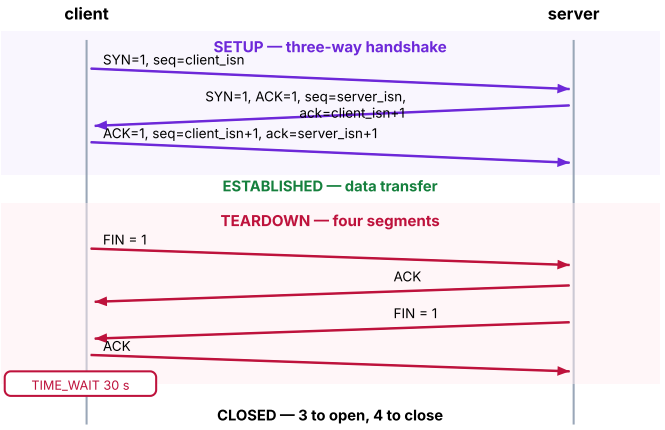
1. **SYN.** Client sends SYN = 1 with a randomly chosen `client_isn`. No data.
   Significance: asks to open the connection and tells the server **where the client's byte numbering starts**.
2. **SYN + ACK.** Server replies SYN = 1, its own `server_isn`, `ack = client_isn + 1`, and **allocates buffers**.
   Significance: agrees to connect, **confirms the client's number** and **announces its own**.
3. **ACK.** Client sends SYN = 0, `seq = client_isn + 1`, `ack = server_isn + 1`, allocates buffers; **may already carry data**.
   Significance: confirms the server's number — connection is **ESTABLISHED**.

Why three: each side must **announce its own initial sequence number** and have it **acknowledged by the other**. Two steps cannot do both.

B. Termination — 4 segments

1. **FIN** from the client (no more data to send)
2. **ACK** from the server — the client→server direction is now closed
3. **FIN** from the server (it has finished too)
4. **ACK** from the client — resources deallocated at **both** ends

Why four: TCP is **full-duplex**, so each direction is shut down **separately**. The client first waits in **TIME_WAIT (~30 s)** so a lost final ACK can be re-sent and old segments can die out.



IF IT SAYS "STATES" INSTEAD Draw the ring of **6 client states** and label every arrow *event / action*: CLOSED →(connect, send SYN)→ SYN_SENT →(rcv SYN+ACK, send ACK)→ **ESTABLISHED** →(app closes, send FIN)→ FIN_WAIT_1 →(rcv ACK)→ FIN_WAIT_2 →(rcv FIN, send ACK)→ TIME_WAIT →(30 s)→ CLOSED. First three = setup, last four = termination.

8 Flow control and the receive window (rwnd)

CO2 | CIE-1 MAY 2024 & JUN 2023 | 5 MARKS

Problem: if the network layer delivers data faster than the application reads it, the **receive buffer fills and overflows** — data is lost.

Solution: TCP **flow control** — a **speed-matching service** that matches the sender's rate to the **receiving application's reading rate**.

The receiver keeps `RcvBuffer` plus **LastByteRead** and **LastByteRcvd**. To avoid overflow:

$$\texttt{LastByteRcvd} - \texttt{LastByteRead} \le \texttt{RcvBuffer}$$

$$\texttt{rwnd} = \texttt{RcvBuffer} - [\texttt{LastByteRcvd} - \texttt{LastByteRead}]$$

$$\texttt{LastByteSent} - \texttt{LastByteAcked} \le \texttt{rwnd}$$

The receiver advertises **rwnd in the receive-window field of every ACK**. When **rwnd = 0** the sender stops but keeps sending **small probe segments** so it learns when space frees up.

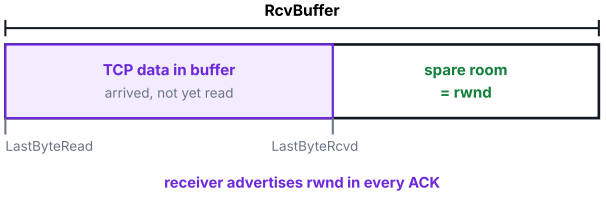


Fig. 3.38

WORKED EXAMPLE RcvBuffer = **4096 B**, LastByteRcvd = **3000**, LastByteRead = **1000**.
rwnd = 4096 − (3000 − 1000) = **2096 B** — the sender may have at most 2096 bytes unACKed. If the application then reads up to byte 3000, rwnd rises back to **4096 B**.

HOOK **rwnd protects the receiver, cwnd protects the network.** Data actually in flight = **min(cwnd, rwnd)**.

9 CIDR — find the mask, N, first and last address

CO3 · CAME IN CIE-1 APR 2025 · 4–5 MARKS · CANNOT BE ARGUED WITH

THE 4 LINES THAT SOLVE EVERY ADDRESSING QUESTION **$N = 2^{32-n}$** · mask = **n ones then (32−n) zeros** · **first = address AND mask** · **last = address OR NOT(mask)**. Also: N = NOT(mask) + 1, and $n = 32 - \log_2 N$.

192.168.10.55/26 — the Apr 2025 question

```
(i) number of addresses
host bits = 32 − 26 = 6
N = 2⁶ = 64 addresses in the block
usable  = 64 − 2 = 62
          (network + broadcast excluded)

(ii) mask and first address
/26 = 11111111.11111111.11111111.11000000
    = 255.255.255.192

last octet 55 = 00110111
keep top 2 bits, zero the rest:
   00|110111  →  00|000000 = 0
first (network)  = 192.168.10.0
set the 6 host bits to 1 → 00111111 = 63
last (broadcast) = 192.168.10.63
range = 192.168.10.0 – 192.168.10.63
```

156.180.150.89/26 — by the mask method

```
mask      = 255.255.255.192
NOT(mask) = 0.0.0.63
N = 63 + 1 = 64

first = address AND mask
   89  = 01011001
   192 = 11000000
   AND = 01000000 = 64  → 156.180.150.64

last = address OR NOT(mask)
   89  = 01011001
   63  = 00111111
   OR  = 01111111 = 127 → 156.180.150.127
```

Prefix	/25	/26	/27	/28	/29	/30
Mask last octet	128	**192**	224	**240**	248	252
Addresses	128	64	32	16	8	4
Usable	126	62	30	14	6	2

TRAP "Total number of **valid** IP addresses" is ambiguous — **write both**: "64 in the block, of which 62 are usable host addresses."

10 Subnetting design — unequal sub-blocks from one granted block

CO3 · MOST-ASKED UNIT 3 TOPIC — 5 PAST QUESTIONS · 6–9 MARKS

> *"An organization is granted a block with the beginning address **14.24.74.0/24**. It needs three subblocks: one of **10** addresses, one of **60** and one of **120**. Design the sub-blocks."*

Step 1 — the given block

n = 24 → N = $2^{32-24} = 2^8$ = **256 addresses**: **14.24.74.0 to 14.24.74.255**.

Step 2 — the 3 rules (write them, they carry marks)

1. Each subnet size must be a **power of 2** — round every requirement **up**.
2. **$n_{sub} = 32 - \log_2 N_{sub}$**
3. Each **start address must be divisible by its own size** — which happens automatically if you **allocate the largest first**.

Step 3 — allocation, largest first (120 → 60 → 10)

Sub	Need → size	Prefix	Mask	First	Last
1	120 → **128**	**/25**	255.255.255.128	14.24.74.0	14.24.74.127
2	60 → **64**	**/26**	255.255.255.192	14.24.74.128	14.24.74.191
3	10 → **16**	**/28**	255.255.255.240	14.24.74.192	14.24.74.207

Last address = first + N_{sub} − 1 (128 + 64 − 1 = 191, 192 + 16 − 1 = 207).

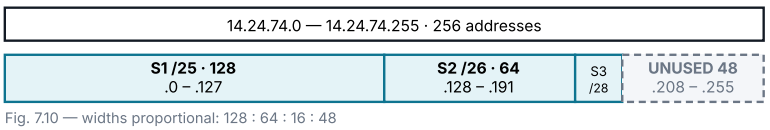


Fig. 7.10 — widths proportional: 128 : 64 : 16 : 48

Step 4 — leftover

Used = 128 + 64 + 16 = **208**. Unused = 256 − 208 = **48 addresses: 14.24.74.208 – 14.24.74.255**.

HOOK **"Biggest first, or the maths breaks."** Round-up table: **10→16 · 50→64 · 60→64 · 120→128 · 250→256 · 500→512 · 1000→1024**.

EQUAL-SUBNETS VARIANT "Four equal subnets from **128.119.40.64/26**": /26 = 64 addresses → 64 / 4 = **16 each** → borrow **2 bits** → **/28**, mask 255.255.255.240. Starts step by 16: **.64, .80, .96, .112**; each has **14 usable hosts**.

How to spend whatever time you have left

READ THIS LAST

Study order

First 25 min — Q9 and Q10 (pure method, guaranteed marks), then Q5 (the three formulas), then Q4's diagram. That is roughly **12 marks of near-certainty**.

Then — Q2, Q6, Q7. All three are "list the parts" answers you can scaffold quickly.

Last — Q1, Q3, Q8.

Three rules in the hall

1. **Numericals first**, inside whichever of Q2 / Q3 you pick — and pick based on the **numerical**, not the theory.
2. **Draw before you write.** "Illustrate / Show / Draw" is about **2 marks of picture**. A labelled diagram alone beats a paragraph with none.
3. **Number your points** — "4 reasons", "3 phases", "6 steps". The marking key is written against lists.

Didn't make the 10 — 10 spare minutes each

SMTP, A to B (was Apr 2025 Q3b): user agent → A's mail server (**outgoing queue**) → SMTP client opens **TCP port 25** to B's server → handshaking → message sent → placed in **B's mailbox** → B reads it with **POP3 110 / IMAP 143**. SMTP goes **directly**, no intermediate servers.

Mesh vs star (in three CIE-1s): links = **$n(n-1)/2 = 10$** for $n = 5$, ports = $n-1 = 4$. Mesh: no single point of failure, expensive. Star: hub fails → whole network down, cheap and easy.

Go. You've got this.